

Aula prática N.º 7

Objetivos

- Familiarização com o modo de funcionamento de um periférico com capacidade de produzir informação.
- Utilização da técnica de interrupção para detetar a ocorrência de um evento e efetuar o consequente processamento.
- Efetuar a conversão analógica/digital de um sinal de entrada e mostrar o resultado no sistema de visualização implementado anteriormente.

Introdução

O Sistema de Interrupções do PIC32

O microcontrolador PIC32 dispõe de um sistema de interrupções flexível, que pode operar em dois modos distintos:

- **Single-vector mode** – utiliza apenas um único vetor (vetor 0) para todas as fontes de interrupção. Neste modo, a identificação da fonte que originou a interrupção é feita por software, na Rotina de Serviço à Interrupção (RSI).
- **Multi-vector mode** – utiliza vetores definidos pelo fabricante, estando cada vetor associado a uma fonte específica de interrupção. Este é o modo configurado na placa DETPIC32.

O sistema de interrupções é gerido por um controlador de interrupções externo ao CPU. Este módulo suporta até 96 fontes de interrupção (75 no caso do PIC32MX7xx), incluindo 5 fontes externas configuráveis por transição ativa (subida ou descida). Estão disponíveis até 64 vetores de interrupção (51 no PIC32MX7xx).

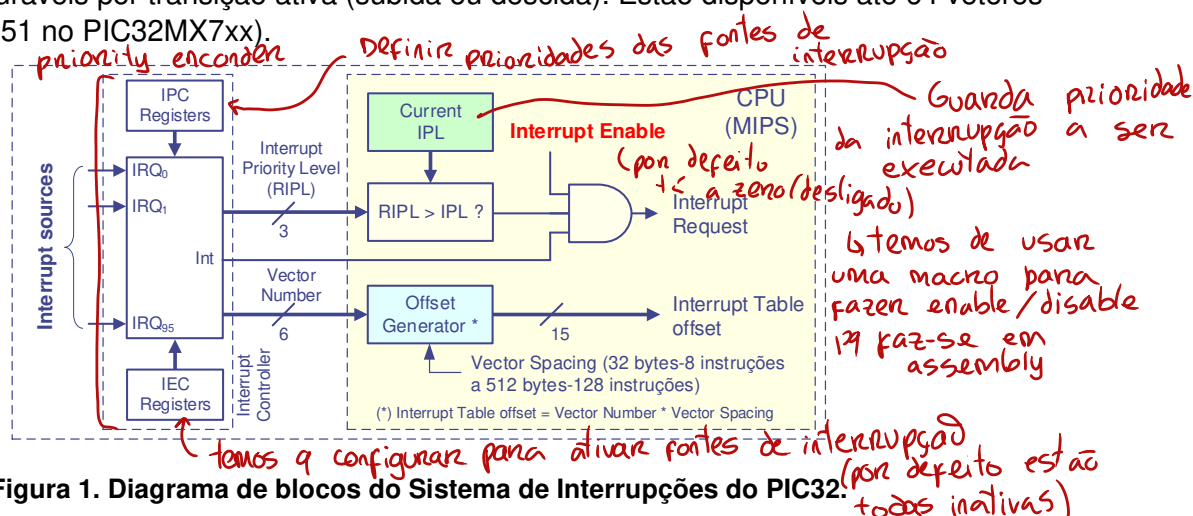


Figura 1. Diagrama de blocos do Sistema de Interrupções do PIC32.

O controlador de interrupções permite atribuir diferentes níveis de prioridade a cada fonte, funcionando como um *priority encoder*, que seleciona e encaminha para o CPU o pedido pendente com maior prioridade, identificado simultaneamente pelo número do vetor e pelo nível de prioridade (RIPL).

Para que o CPU reconheça interrupções, o sinal *Interrupt Enable* tem que estar ativo, o que é feito através da instrução **EI** (por defeito, este sinal encontra-se desativado). Uma RSI associada a uma interrupção com prioridade **IPL** pode ser interrompida por outra interrupção com prioridade **RIPL**, desde que **RIPL > IPL**.

As fontes de interrupção podem ser classificadas em:

- **Internas** - até 91 fontes, geradas pelos periféricos internos do microcontrolador;
- **Externas** - 5 entradas externas configuráveis por transição ativa.

Tabela de Vetores (*Multi-vector Mode*)

No modo *multi-vector*, a tabela de vetores é inicializada com instruções de salto para as respetivas Rotinas de Serviço à Interrupção (RSI). Quando ocorre uma interrupção, o CPU calcula o endereço da entrada correspondente na tabela de vetores e executa um salto para esse endereço. Cada entrada da tabela contém, tipicamente, uma instrução de salto incondicional para a RSI associada.

O endereço base da tabela de vetores de interrupção (*Interrupt Vector Table*) e o valor do *vector spacing* são configuráveis por software. O *vector spacing* pode ser configurado com um dos valores 32, 64, 128, 256 ou 512 bytes.

Na placa DETPIC32, o endereço inicial da tabela de vetores (vetor 0) é **0x9D000200** e o *vector spacing* é 32.

Exemplo: para o vetor 2, com **base_address=0x9D000200** e espaçamento de 32, o endereço correspondente será **0x9D000240** ($0x9D000200 + 2 * 0x20$).

Registos do Sistema de Interrupções

O sistema de interrupções utiliza os seguintes registos:

- **IECx** – *Interrupt Enable Control Registers*
Permitem ativar ou desativar cada fonte de interrupção.
- **IPCx** – *Interrupt Priority Control Registers*
Definem o nível de prioridade das fontes de interrupção, utilizando 3 bits (valores de 0 a 7, sendo 0 equivalente a interrupção desativada).
- **IFSx** – *Interrupt Flag Status Registers*
Contêm as *flags* que sinalizam a ocorrência de uma interrupção. A ativação da *flag* é automática, mas a sua limpeza é da responsabilidade do programador.
- **INTCON** – *Interrupt Control Register* – Para interrupções externas (Porto D)
Permite configurar a polaridade da transição ativa das interrupções externas (*rising edge* ou *falling edge*).

Cada fonte de interrupção possui bits associados nos registos **IEC**, **IPC** e **IFS**:

- **Interrupt Enable (xxxIE)** – ativa/desativa a interrupção. *nome do periférico → Para o ADC: AD1IE*
- **Priority Level (xxxIP)** – define nível de prioridade (1 a 7).
- **Interrupt Flag (xxxIF)** – indica a ocorrência da interrupção e deve ser manualmente reposta a 0 no programa.

O identificador **xxx** corresponde ao nome do periférico, conforme definido pelo fabricante e apresentado no manual **PIC32MX5XX/6XX/7XX, Family Data Sheet**, na tabela 7-1 (páginas 74 a 76). Nessa tabela é igualmente indicada a correspondência entre cada fonte de interrupção e os respetivos registos IEC, IPC e IFS.

Exemplo para o conversor A/D:

TABLE 7-1: INTERRUPT IRQ, VECTOR AND BIT LOCATION (CONTINUED)

Interrupt Source ⁽¹⁾	IRQ Number	Vector Number	Interrupt Bit Location			
			Flag	Enable	Priority	Sub-Priority
AD1 – ADC1 Convert Done	33	27	IFS1<1>	IEC1<1>	IPC6<28:26>	IPC6<25:24>

Nome do periférico: **AD1**

Vetor: 27

Registos associados: **IFS1** (bit **AD1IF**), **IEC1** (bit **AD1IE**), **IPC6** (bits **AD1IP**)

Utilização do conversor A/D com interrupções

Como foi já observado na aula prática anterior, quando o módulo A/D termina uma sequência de conversão gera um pedido de interrupção (ativa o bit **AD1IF** do registo **IFS1**). Para que este pedido de interrupção tenha seguimento, o sistema de interrupções do microcontrolador tem de estar devidamente configurado, de modo a garantir que, na ocorrência do evento de fim de conversão, a rotina de serviço à interrupção (RSI) é executada.

Para isso, para além das configurações do módulo A/D, já efetuadas anteriormente, é ainda necessário configurar o sistema de interrupções, procedendo do seguinte modo:

1. configurar o nível de prioridade das interrupções geradas pelo módulo A/D – registo **IPC6**¹, nos 3 bits **AD1IP**; deve ser escrito um valor entre 1 e 6; o valor 7, correspondente à prioridade máxima, não deve ser utilizado; o valor 0 faz com que os pedidos de interrupção nunca sejam aceites, equivalendo à desativação desta fonte de interrupção:

```
IPC6bits.AD1IP = 2; // configure priority of A/D interrupts
```

2. fazer o *reset* de alguma interrupção pendente – registo **IFS1**, bit **AD1IF**:

```
IFS1bits.AD1IF = 0; // clear A/D interrupt flag
```

3. autorizar as interrupções geradas pelo módulo A/D – registo **IEC1**, bit **AD1IE**:

```
IEC1bits.AD1IE = 1; // enable A/D interrupts
```

4. ativar globalmente o sistema de interrupções no CPU:

```
EnableInterrupts(); // Macro defined in "detpic32.h"
```

A Rotina de Serviço à Interrupção terá a seguinte estrutura:

```
// Interrupt service routine (interrupt handler)
void int_(VECTOR) isr_adc(void)
{
    // ISR actions
    (...)
    IFS1bits.AD1IF = 0; // Reset AD1IF flag
}
```

Indica ao compilador que é um IRSA (para ser adicionado prólogo e epílogo)

Continua ação da IRSA

O prefixo "int_" indica ao compilador que se trata de uma rotina de serviço a uma interrupção. Com essa indicação, o compilador gera o código necessário para salvaguardar os registos que são usados pela RSI (*prolog*), repor o valor desses registos no final da execução (*epilog*) e terminar a rotina com a instrução **eret**.

O identificador **VECTOR** deve ser substituído pelo número do vetor associado ao módulo A/D, conforme indicado no **PIC32MX5XX/6XX/7XX, Family Data Sheet** (páginas 74 a 76). Com essa informação, o compilador preenche a tabela de vetores de interrupção (*interrupt vector table*) com uma instrução de salto incondicional para o endereço da primeira instrução da RSI (na posição da tabela correspondente ao vetor).

A limpeza da flag **AD1IF** no final da RSI é obrigatória; caso contrário, a RSI será imediatamente reexecutada.

¹ A informação relativa a cada fonte de interrupção, nomeadamente o vetor associado e registos de configuração, está condensada na tabela das páginas 74 a 76 do PIC32MX5XX/6XX/7XX, Family Data Sheet (disponível no *moodle* de AC2).

Trabalho a realizar

No trabalho prático anterior fez-se a deteção do evento de fim de conversão do módulo A/D (ADC) por *polling*, isto é, num ciclo que espera pela passagem a 1 do bit **AD1IF**. O que se pretende agora é que o atendimento ao evento de fim de conversão seja feito por interrupção e não por *polling*.

1. O programa-esqueleto apresentado de seguida mostra a estrutura-base do programa para interagir com a ADC utilizando interrupções, ainda numa versão simplificada.

Na função **main()**, após as configurações iniciais, é dada ordem de conversão à ADC, seguindo-se a entrada do programa num ciclo infinito. Quando a ADC termina a conversão, gera uma interrupção e o programa salta para a respetiva RSI. Nessa rotina, o resultado da conversão é lido e impresso.

Antes de terminar, a RSI dá nova ordem de conversão à ADC e regressa ao ciclo infinito do programa principal. Sempre que uma conversão termina, é gerado um novo pedido de interrupção e o processo repete-se, caracterizando um funcionamento em *free run mode*.

Neste primeiro exercício pretende-se que a ADC gere a interrupção ao fim de 1 conversão (**SMPI=0**).

```
int main(void)
{
    // Configure all (digital I/O, analog input, A/D module)
    // Configure interrupt system
    EnableInterrupts();           // Global Interrupt Enable
    // Start A/D conversion
    while(1) ← o faz nada, o programa sai quando houver interrupção
    {
        // all activity is processed by the ISR
    }
    return 0;
}
```

A rotina de serviço à interrupção para interação com o módulo A/D e impressão do valor lido poderá ter a seguinte organização:

```
// Interrupt Handler

void _int_(VECTOR) isr_adc(void) // Replace VECTOR by the A/D vector
                                // number - see "PIC32 family data
                                // sheet" (pages 74-76)
{
    // Read conversion result (ADC1BUF0) and print it
    // Start A/D conversion
    IFS1bits.AD1IF = 0;           // Reset AD1IF flag
}
```

Guarde esta versão do seu programa para que possa ser usada na parte 2 deste guião.

2. Integre no programa anterior o sistema de visualização em displays de 7 segmentos. Faça as alterações que permitam a visualização do valor da tensão nos *displays* de 7 segmentos, em decimal.

O programa deverá cumprir os seguintes requisitos:

- a) efetuar 5 sequências de conversão A/D por segundo (frequência de amostragem de 5 Hz), sendo cada sequência constituída por 8 amostras;
- b) enviar informação para o sistema de visualização a cada 10 ms (frequência de refrescamento de 100 Hz).

Note que a frequência de amostragem da ADC e a frequência de refrescamento do sistema de visualização são diferentes e devem ser tratadas de forma independente.

Na organização do código, utilize o programa-esqueleto que se apresenta de seguida:

```
volatile unsigned char voltage = 0;    // Global variable

int main(void)
{
    unsigned int cnt = 0;
    // Configure all (digital I/O, analog input, A/D module, interrupts)
    ...
    EnableInterrupts(); // Global Interrupt Enable
    while(1)
    {
        if(cnt == 0)        // 0, 200 ms, 400 ms, ... (5 samples/second)
        {
            // Start A/D conversion
        }
        // Send "voltage" value to displays
        cnt = (cnt + 1) % ??;
        // Wait ?? ms
    }
    return 0;
}

void _int_(VECTOR) isr_adc(void)
{
    // Read 8 samples (ADC1BUF0, ..., ADC1BUF7) and calculate average
    // Calculate voltage amplitude
    // Convert voltage amplitude to decimal and store the result in the
    // global variable "voltage"
    // Reset AD1IF flag
}
```

Notas:

1. A palavra-chave **volatile** dá a indicação ao compilador que a variável pode ser alterada de forma não explicitada na zona de código onde está a ser usada (i.e., noutra zona de código, como por exemplo numa RSI). Com esta palavra-chave força-se o compilador a, sempre que o valor da variável seja necessário, fazer o acesso à posição de memória onde essa variável reside, em vez de usar uma eventual cópia, potencialmente com um valor desatualizado, residente num registo interno do CPU.
2. Deve-se evitar a execução de operações demoradas no interior das RSI. As RSI devem ser tão curtas quanto possível, limitando-se ao tratamento estritamente necessário do evento que originou a interrupção. Operações mais demoradas podem afetar a capacidade de resposta do sistema a outros eventos e comprometer o cumprimento dos requisitos temporais (aumento da latência de resposta a outras fontes de interrupção).
3. Para a validação do funcionamento podem ser utilizados pontos de teste simples, como por exemplo LEDs ou pequenas mensagens de depuração, para confirmar se as diferentes tarefas estão a ser executadas com a periodicidade esperada.

(continua na página seguinte)

Parte II

Na aula prática anterior foi medido o tempo de conversão do conversor A/D. Conhecendo esse valor, é agora possível estimar o intervalo de tempo que decorre desde o pedido de interrupção gerado pela ADC, até à execução da primeira instrução efetiva da RSI. Este intervalo inclui a latência da interrupção e o tempo de execução do prólogo da RSI.

Para isso, será novamente utilizado o porto digital **RD11** configurado como saída (disponível no ponto de teste **INT4**).

1. Retome o programa que guardou no final do exercício 1 da parte 1 e faça as seguintes alterações:
 - a) No programa principal configure o porto **RD11** como saída;
 - b) Na RSI desative o porto **RD11** no início e ative-o imediatamente antes de dar ordem de início de conversão ao conversor A/D;
 - c) Remova a chamada à *system call* de impressão do valor lido (substitua-o por simples leitura para a variável "**adc_value**", declarada como **volatile**).

```
void _int_(VECTOR) isr_adc(void)
{
    volatile int adc_value;
    // Reset RD11 (LATD11 = 0)
    // Read ADC1BUF0 value to "adc_value"
    // Set RD11 (LATD11 = 1)
    // Start A/D conversion
    // Reset AD1IF flag
}
```

Execute o programa e, utilizando um osciloscópio, meça o intervalo de tempo durante o qual o porto **RD11** permanece ao nível lógico 1 anote esse valor. Subtraindo a esse tempo o tempo de conversão medido no trabalho prático anterior, obtém-se o tempo que decorre desde o pedido de interrupção até à execução da primeira instrução efetiva da RSI, designado na Figura 2 por t_{LP} .

Sabendo que a frequência do CPU é 40 MHz, poderá explicitar o resultado em ciclos de relógio, utilizando a expressão: $\#cycles = t_{LP} \times 40 \times 10^6$.

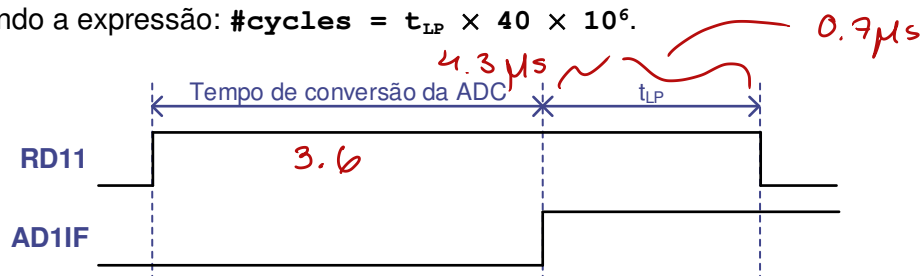


Figura 2. Medição do tempo que decorre desde o pedido de interrupção até à execução da primeira instrução útil da RSI.

2. Pretende-se agora estimar o *overhead* global associado ao atendimento de uma interrupção no PIC32. Para esse efeito, é ainda necessário considerar o tempo correspondente ao regresso ao programa interrompido, essencialmente determinado pelo tempo de execução do epílogo (reposição do contexto salvaguardado no início da RSI).

Para medir esse tempo, coloque o porto de saída **RD11** ao nível lógico 1 no final da RSI (como última instrução da rotina) e volte a colocá-lo ao nível lógico 0 no ciclo infinito do programa principal. Meça, com o osciloscópio, o intervalo de tempo durante o qual o porto **RD11** permanece ativo e expresse esse valor em número de ciclos de relógio. Adicionando esse valor ao obtido no ponto anterior, obtém-se uma boa estimativa do *overhead* global no atendimento de uma interrupção no PIC32.

```
void _int_(VECTOR) isr_adc(void)
{
    ...
    // Reset AD1IF flag
    // Set RD11      (LATD11 = 1)
}

int main(void)
{
    ...
    while(1)
    {
        // Reset RD11  (LATD11 = 0)
    }
    return 0;
}
```

Elementos de apoio

- Slides das aulas teóricas
- PIC32 Family Reference Manual, Section 17 – A/D Module.
- PIC32 Family Reference Manual, Section 08 – Interrupts.
- PIC32MX5XX/6XX/7XX, Family Data Sheet, Pág. 74 a 76.

PDF criado em 14/04/2026